



Web Development Lab

Footer, CSS Grid and Flexbox— Practical Session Guide

15 Guided Steps • Beginner–Intermediate • Estimated Time: 50 mins

Lab Overview

In this session you will build a **modern Footer Section** using a combination of **CSS Grid (for layout)** and **Flexbox (for internal alignment)**.

You will learn how to:

- Structure a multi-column footer
- Use **Grid for layout distribution**
- Use **Flexbox inside sections**
- Style typography and spacing

Each step builds progressively so you can **see how the footer takes shape in real-time**.



Note: Continue working in your existing **index.html** file.

Prerequisites

- Completed Navbar Lab 
- Completed Hero Section Lab 
- Completed Grid (Cards) Lab 
- Basic knowledge of Flexbox and Grid

1. Add the Footer HTML Structure

Objective: Create the structure of the footer links section.

Background

The footer is divided into:

- Brand section
- Product links
- Company links
- Legal links

Each section is placed inside a parent container for layout control.

Instructions

1. Add the footer HTML below your previous sections

Code Snippet

```
<section class="footerLinks">
  <div class="fieldwork-brand">
    <h2>Field Work</h2>
    <p>Professional field documentation for modern <br/>construction
teams.</p>
  </div>

  <div class="product">
    <p>Product</p>
    <ul>
      <li>Features</li>
      <li>Pricing</li>
      <li>Updates</li>
    </ul>
  </div>

  <div class="company">
    <p>Company</p>
    <ul>
      <li>About</li>
      <li>Careers</li>
      <li>Contact</li>
    </ul>
  </div>

  <div class="legal">
    <p>Legal</p>
    <ul>
      <li>Privacy Policy</li>
      <li>Terms</li>
    </ul>
  </div>
</section>
```



Tip: Footers usually group related navigation links.

2. Add Bottom Footer Section

Objective: Create the copyright section.

🕒 3 min

Background

The `<footer>` element is used for **end-of-page content** like copyright.

Instructions

1. Add footer below links section

Code Snippet

```
<footer class="footer">
  <div class="footerContent">
    <p>&copy; 2026 Field Work. All rights reserved.</p>
  </div>
</footer>
```



Tip: Always use semantic HTML elements like `<footer>`.

3. Convert Footer Links into Grid Layout

Objective: Use CSS Grid for multi-column layout.

🕒 6 min

Background

1. CSS Grid allows us to create **multiple columns easily**.
 - `display: grid;`
2. Turns `.footerLinks` into a grid container
 - `grid-template-columns: repeat(4, 1fr);`
 -
3. Creates 4 equal columns
Each column takes **1 fraction of available space**
 - `align-items: center;`
 -
4. Aligns all items vertically

Instructions

1. Apply grid layout

Code Snippet

```
.footerLinks {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  align-items:center;  
}
```



Tip: Grid is perfect for footer layouts.

4. Add Background and Spacing

Objective: Improve layout spacing and appearance.

🕒 5 min

Background

background-color: #FFFFFF;

1. Sets background color
 - a. padding-top: 49px;
2. Adds space inside the top of the footer
 - a. margin: 0px 100px;
3. Adds spacing on left and right

Instructions

1. Add spacing

```
Code Snippet
.footerLinks {
  background-color: #FFFFFF;
  padding-top: 49px;
  margin: 0px 100px;
}
```



Tip: Padding improves internal spacing.

5. Style the Brand Section

Objective: Highlight the company identity.

🕒 5 min

Background

font-size: 20px;

1. Controls heading size
 - a. font-weight: bold;
2. Makes text prominent
 - a. letter-spacing: -0.5px;
3. Adjusts spacing between letters

Instructions

1. Style brand heading

Code Snippet

```
.fieldwork-brand h2{  
  color:#1A1A1A;  
  font-weight: bold;  
  line-height: 30px;  
  letter-spacing: -0.5px;  
  font-size: 20px;  
  margin: 0px;  
}
```

💡 **Tip: Brand section should stand out.**

6. Use Flexbox for Footer Columns

Objective: Arrange items vertically inside columns.

🕒 6 min

Background

1. **display: flex;**
 - Turns container into flexbox
2. **flex-direction: column;**
 - Stacks items vertically
3. **gap: 16px;**
 - Adds spacing between items

Instructions

1. Apply Flexbox to columns

Code Snippet

```
.product, .company, .legal{  
  display: flex;  
  flex-direction: column;  
  gap: 16px;  
}
```



Tip: Grid handles layout, Flexbox handles internal alignment.

7. Style Column Titles

Objective: Improve readability of headings.

🕒 5 min

Background

1. **font-size: 14px;**
 - Controls text size
2. **font-weight: bold;**
 - Makes headings stand out
3. **margin: 0;**
 - Removes default spacing

Instructions

1. Style titles

Code Snippet

```
.product p, .company p, .legal p{
  color:#1A1A1A;
  font-size: 14px;
  line-height: 21px;
  margin: 0;
  font-weight: bold;
}
```



Tip: Remove default margins for better control.

8. Style List Items

Objective: Clean up list styling.

🕒 5 min

Background

1. **list-style-type: none;**
 - a. Removes bullet points
2. **padding-left: 0px;**
 - a. Removes default indentation
3. **color: #6B6B6B;**
 - a. Applies softer text color

Instructions

1. Style lists

Code Snippet

```
.product ul, .company ul, .legal ul{  
  color:#6B6B6B;  
  font-size: 14px;  
  line-height: 21px;  
  margin: 0px 0px 8px;  
  list-style-type: none;  
  padding-left: 0px;  
}
```



Tip: Footer links are usually subtle (not bold).

9. Understand Grid Column Distribution

Objective: Understand layout behavior.

🕒 4 min

Background

1. **repeat(4, 1fr);**
 - a. 4 → number of columns
 - b. 1fr → equal width distribution
2. **Each column:**
 - a. Takes equal space
 - b. Adjusts automatically



Tip: **fr** units are powerful for responsive layouts.

10. Align Content Vertically

Objective: Ensure consistent alignment.

🕒 4 min

Background

1. **align-items: center;**
 - Aligns all grid items vertically

Instructions

1. Observe alignment



Tip: Alignment improves visual balance.

11. Style Bottom Footer Section

Objective: Design the copyright area.

🕒 5 min

Background

1. **text-align: center;**
 - a. Centers text
2. **padding: 20px 0;**
 - a. Adds vertical spacing
3. **color: #6B6B6B;**
 - a. Applies muted text color

Instructions

1. Style `.footer`

Code Snippet

```
.footer {  
  background-color: #FFFFFF;  
  color: #6B6B6B;  
  text-align: center;  
  padding: 20px 0;  
  line-height: 21px;  
  font-size: 14px;  
}
```



Tip: Footer text should be subtle, not dominant.

12. Understand Grid + Flexbox Combination

Objective: Reinforce layout principles.

🕒 5 min

Background

- Grid → divides page into columns
- Flexbox → organizes content inside columns

This hybrid approach is:

- Scalable
- Clean
- Industry standard



Tip: This is how real-world layouts are built.

13. Improve Spacing and Readability

Objective: Fine-tune layout spacing.

🕒 4 min

Background

Spacing tools used:

- margin → outside spacing
- padding → inside spacing
- gap → between elements

Instructions

1. Review spacing

💡 **Tip: Good spacing = professional design.**

Layout Observation

Objective: Analyze the final structure.

🕒 4 min

Background

You now have:

- 4-column grid layout
- Structured footer links
- Bottom footer section

Observe:

- Alignment
- Spacing
- Readability

15. Final Review and Testing

Objective: Validate the footer section.

🕒 8 min

Instructions

1. Refresh browser
2. Check alignment

Final Check

- Grid layout working correctly
- Columns aligned properly
- No CSS errors
- Footer displays cleanly